Analyze Cache Coherence (MSI, MESI, MOESI) and NUMA Memory Access Performance

PGC Experiment 04 - Lab Report

1. Objective

To study how the MSI, MESI and MOESI cache coherence protocols keep the caches of a multi-core processor consistent, and to evaluate the performance effect of coherence traffic (false sharing) and of NUMA memory placement (local vs remote memory). One example is used throughout: two threads, each incrementing its own counter, with the two counters placed either on the same 64-byte cache line (*adjacent*) or on separate cache lines (*padded*). The experiment measures correctness, execution time and scalability as the number of threads grows.

2. Theory in Brief

Coherence and false sharing. Each core has a private cache, so a write by one core makes other copies of the same line stale. A snooping protocol makes every cache watch the bus and invalidate or update its copy. Coherence is tracked per cache line, not per variable: two independent counters on one line make the line bounce between the cores (false sharing). The results stay correct but the traffic hurts speed. Padding each counter to its own line removes the effect.

Protocol	States	Key idea
MSI	M, S, I	Even a line no other cache holds needs a bus upgrade before its first write. A Modified line read by another core is written back to memory.
MESI	M, E, S, I	Exclusive state: a private clean line moves to Modified silently (no bus transaction).
MOESI	M, O, E, S, I	Owned state: the dirty owner supplies the line cache-to-cache, so no write-back to memory is needed.

NUMA. Memory is split between nodes; a core reaches its own node's RAM (local) faster than another node's RAM (remote). `numactl --cpunodebind=N --membind=M` places threads and memory explicitly. On a single physical machine, virtual NUMA nodes are backed by the same RAM, so little or no local/remote difference is expected.

3. Environment and Method

Item	Value
Platform	VMware Workstation guest, Ubuntu 24.04, gcc 13.3.0 with OpenMP
CPU layout	4 vCPUs; node 0 = CPUs 0-1 (1966 MB), node 1 = CPUs 2-3 (1950 MB)
NUMA distances	local 10, remote 20 (virtual nodes)
Initial VM	Only 1 NUMA node: --membind=1 was rejected; virtual NUMA was then enabled in the .vmx file
Workload	100,000,000 increments in total, split evenly over the threads; 512 MB buffer read once per cache line, 4 passes
Repetitions	One run per configuration (the manual suggests the median of three)

Programs: **Program 1** simulator (coherence_sim.py), **Program 2** false_sharing.c, **Program 3** numa_sweep.c, **Program 4** compare_numa.c (Tables 1 and 2). Source code is in the `src/` folder.

4. Results

4.1 Part A - Protocol simulation (1000 iterations, 4000 memory operations)

State of line A (core 0 / core 1) after each operation, adjacent layout, first iteration:

Operation	MSI	MESI	MOESI
core 0 reads its counter	S / I	E / I	E / I
core 0 writes its counter	M / I	M / I	M / I
core 1 reads its counter	S / S	S / S	O / S
core 1 writes its counter	I / M	I / M	I / M

Layout	Protocol	Bus tx	BusRd	RdX	Upgr	Inval.	WB	C2C	Silent E-M
adjacent	MSI	4000	2000	0	2000	1999	1999	0	0
adjacent	MESI	3999	2000	0	1999	1999	1999	0	1
adjacent	MOESI	3999	2000	0	1999	1999	0	3998	1
padded	MSI	4	2	0	2	0	0	0	0
padded	MESI	2	2	0	0	0	0	0	2
padded	MOESI	2	2	0	0	0	0	0	2

Bus tx = BusRd + BusRdX + BusUpgr; WB = forced write-back to memory; C2C = cache-to-cache transfers.

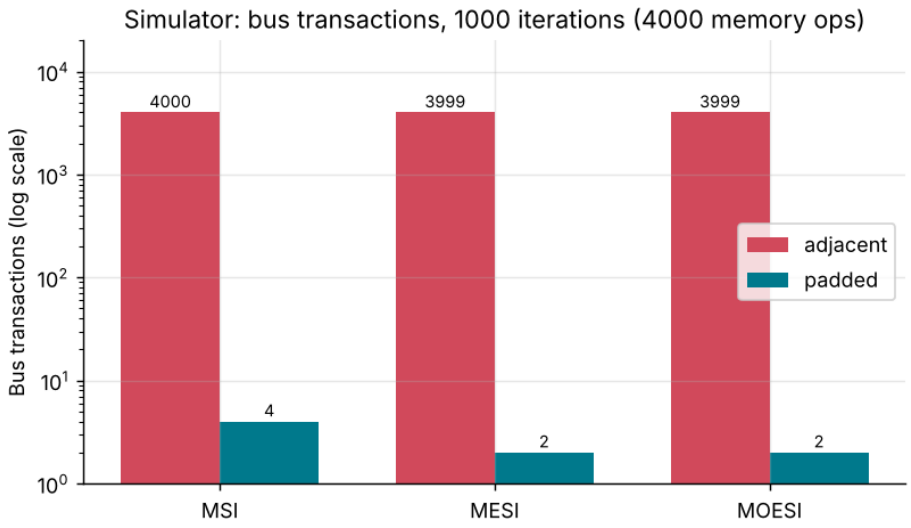


Fig. 1: Bus transactions per protocol and layout (log scale).

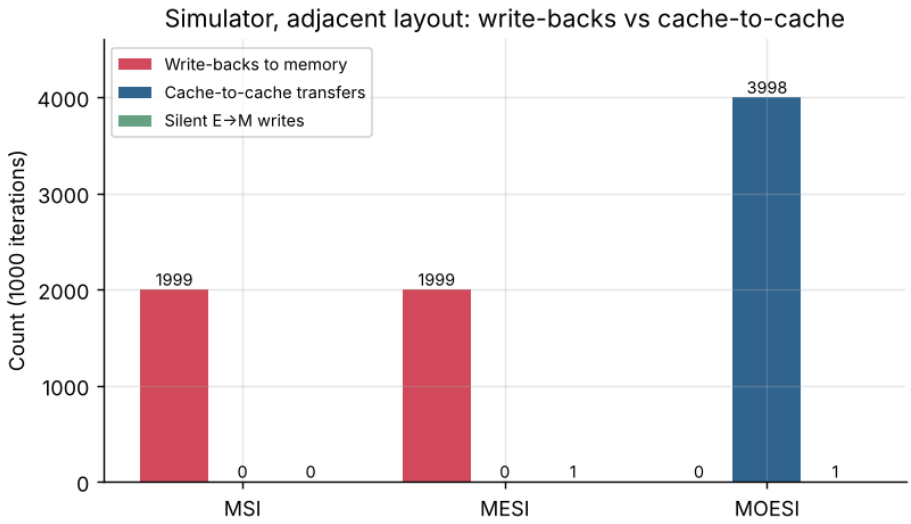


Fig. 2: Adjacent layout - write-backs (MSI, MESI) are replaced by cache-to-cache transfers in MOESI.

4.2 Part B1 - False sharing (Program 2)

With 2 threads, Program 2 measured **adjacent 0.0185 s** and **padded 0.0151 s** (ratio 1.23). Both gave total = 100,000,000.

4.3 Table 1 - Two-counter example (Program 4), time in seconds

Run	Threads	Adjacent	Padded	Adj / Pad	Speed-up adj.	Speed-up pad.
default	1	0.0225	0.0219	1.03	1.00	1.00
default	2	0.0147	0.0110	1.34	1.53	1.99
default	4	0.0095	0.0074	1.28	2.37	2.96
local	1	0.0241	0.0235	1.03	1.00	1.00
local	2	0.0147	0.0139	1.06	1.64	1.69
local	4	0.0154	0.0121	1.27	1.56	1.94
remote	1	0.0228	0.0206	1.11	1.00	1.00
remote	2	0.0149	0.0114	1.31	1.53	1.81
remote	4	0.0145	0.0125	1.16	1.57	1.65

All 18 runs reported Final Count = 100000000 (Correct). 'local' and 'remote' runs used numactl --cpunodebind=0.

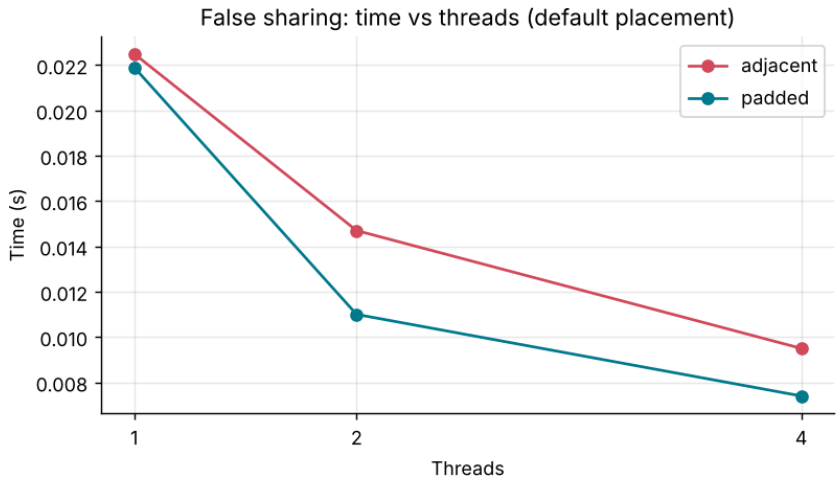


Fig. 3: Execution time vs threads, default placement.

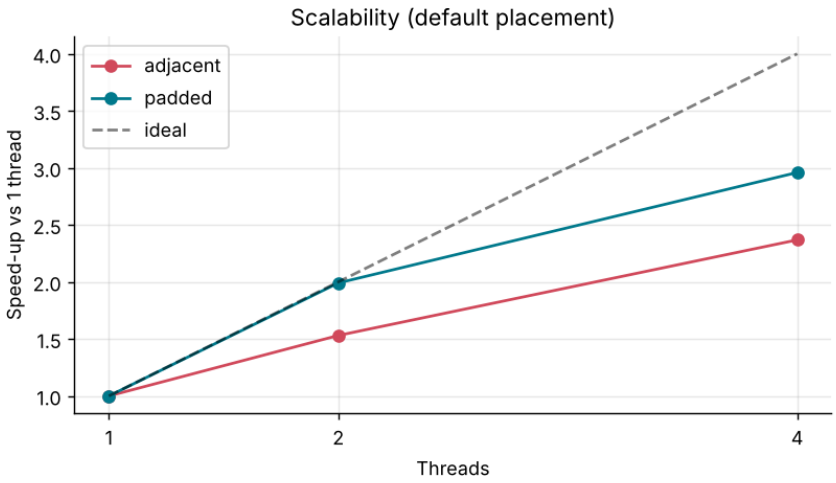


Fig. 4: Speed-up relative to 1 thread, default placement (dashed = ideal).

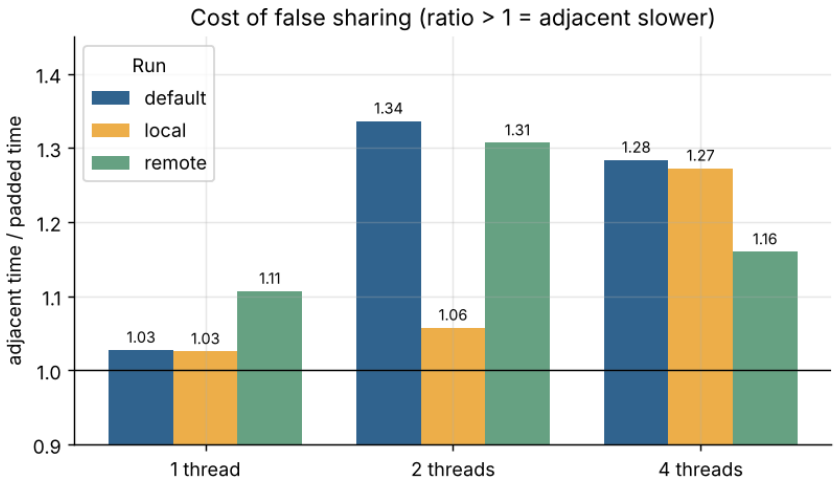


Fig. 5: Adjacent / padded time ratio for each run and thread count.

4.4 Part B2 - NUMA memory sweep

Program 3 (2 threads, 512 MB): local **0.0977 s**, remote **0.0978 s** (ratio 1.00). On the earlier 1-node VM, local was 0.0961 s and the remote run could not start.

Table 2 (Program 4), time in seconds

Threads	Default	Local (--membind=0)	Remote (--membind=1)	Remote / Local
1	0.1072	0.1075	0.1065	0.99
2	0.1010	0.1056	0.0995	0.94

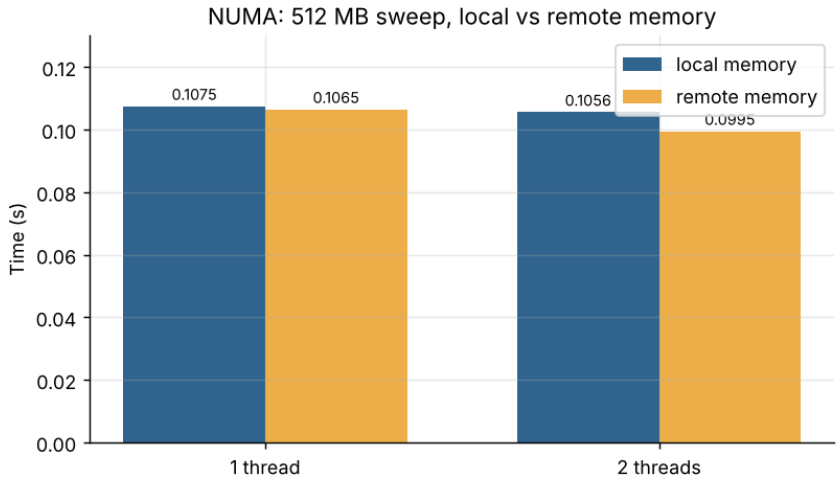


Fig. 6: 512 MB memory sweep, local vs remote memory.

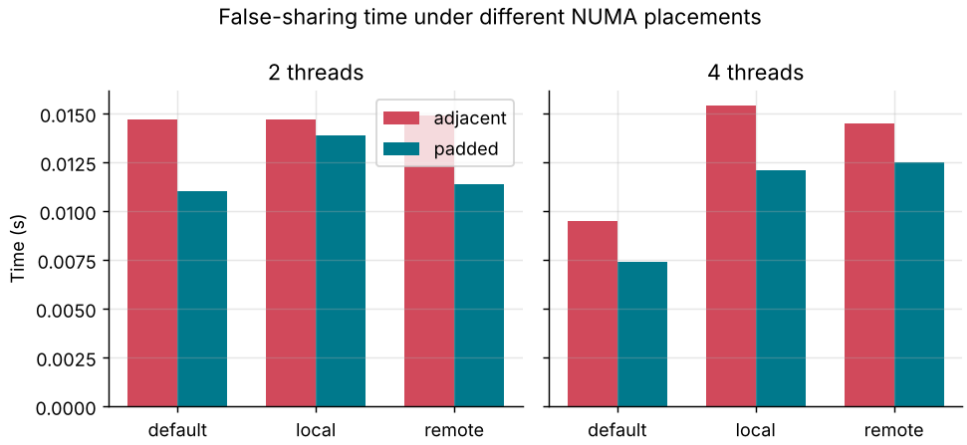


Fig. 7: False-sharing time under default, local and remote placement.

4.5 Results to record (manual Section 9.3)

Run	Time (s)
adjacent, 2 threads (Program 2)	0.0185
padded, 2 threads (Program 2)	0.0151
Table 2, LOCAL, 1 thread	0.1075
Table 2, LOCAL, 2 threads	0.1056
Table 2, REMOTE, 1 thread	0.1065
Table 2, REMOTE, 2 threads	0.0995
adjacent, one thread on each node (optional)	not run

Ratios: adjacent / padded = 1.23 (Program 2); remote / local = 1.00 (Program 3), 0.99 and 0.94 (Table 2, 1 and 2 threads).

5. Observation / Comparison Table (from the measurements)

Case	Correct?	Relative performance (measured)	Scalability (measured)
MSI (simulated)	Yes	Most bus traffic: 4000 tx adjacent; 4 tx padded (upgrade before first write)	Traffic grows with every shared-line access
MESI (simulated)	Yes	3999 tx adjacent; 2 tx padded (silent E to M saves upgrades for private data)	Same ping-pong on a shared line as MSI
MOESI (simulated)	Yes	3999 tx adjacent; 0 write-backs (1999 in MSI/MESI), 3998 cache-to-cache transfers	Lowest memory traffic of the three
Adjacent counters	Yes	Slower than padded in all multi-thread runs: ratio 1.06-1.34	Speed-up 1.53x (2 thr), 2.37x (4 thr), default run
Padded counters	Yes	Fastest: 0.0110 s (2 thr), 0.0074 s (4 thr), default run	Speed-up 1.99x (2 thr), 2.96x (4 thr), default run
Local memory (NUMA)	Yes	Reference: 0.1075 s (1 thr), 0.1056 s (2 thr)	Almost flat from 1 to 2 threads
Remote memory (NUMA)	Yes	0.1065 s (1 thr), 0.0995 s (2 thr): no penalty in the VM	Almost flat from 1 to 2 threads

6. Discussion

Simulator. MESI saves the upgrade for private data (padded: 4 to 2 bus transactions) but, because the two counters share a line, it saves almost nothing in the adjacent layout (3999 vs 4000) - the line ping-pongs under every protocol. MOESI removes all 1999 write-backs to memory by letting the owner supply the line; the cost is 3998 cache-to-cache transfers over the interconnect. (The manual's sample table lists 1999 transfers; the simulator counts the supply on reads and on invalidating writes and reports 3998.)

False sharing. Padded counters were faster whenever more than one thread ran. With one thread there is no sharing, yet the ratio was 1.03-1.11, which shows the noise level of these very short (7-25 ms) timings. The 2- and 4-thread default ratios (1.34, 1.28) and the Program 2 ratio (1.23) lie clearly above that, but the 2-thread local run (1.06) is within noise. The effect is much smaller than the several-fold slowdowns often seen on bare metal; plausible reasons (not verified here) are that the vCPUs share a physical core or cache, the runs are short, and recent CPUs forward stores to loads quickly.

Scalability. In the default run the padded layout scaled almost ideally to 2 threads (1.99x) and reached 2.96x on 4; the adjacent layout reached only 1.53x and 2.37x. In the local and remote runs the 4-thread times do not improve over 2 threads because `--cpunodebind=0` confines the process to CPUs 0-1, so four threads share two vCPUs. Those 4-thread numbers therefore measure oversubscription, not scaling.

NUMA. Local and remote times are equal within noise (ratio 1.00 in Program 3; 0.99 and 0.94 in Table 2, where remote was even slightly faster, which is noise). Both virtual nodes sit on the same physical memory, so no remote penalty is expected. The sweep also barely speeds up from 1 to 2 threads (0.1075 to 0.1056 s), indicating a memory-bound loop. A real local-vs-remote difference requires a multi-socket machine.

7. Limitations

- Each configuration was run once; the median of three runs would reduce noise.

- The 'one thread on each node' optional run (taskset -c 0,2) was not performed.

- The VM hides the physical topology, so the false-sharing penalty and NUMA behaviour are not representative of bare metal.

8. Conclusion

All three protocols kept the data correct; they differ in bus traffic. The simulation shows MESI saving the upgrade transaction for data used by one core (4 to 2 transactions) and MOESI avoiding write-backs to memory (1999 to 0) by letting the owner supply the dirty line. The two-counter experiment confirmed that coherence works on whole cache lines: counters sharing a line were slower than padded counters in every multi-thread run (up to 1.34x) and scaled worse (2.37x vs 2.96x on 4 threads), although the effect was modest in this VM. NUMA placement showed no measurable difference here because the virtual nodes share physical memory; the placement rule (keep threads and their memory on the same node) still applies on real multi-socket hardware. Data layout and placement matter as much as the protocol for the scalability of a parallel program.